Spring Boot — Zero to Master

22 topics, grouped as Beginner → Intermediate → Advanced → Master. Work through the topics in order; type every code example yourself and finish the practice tasks.

Contents

Beginner (6 topics)

01 Introduction to Spring and Spring Boot

02 Project Structure and First Application

03 Inversion of Control and Dependency Injection

04 Beans and Stereotype Annotations

05 REST Controllers

06 Configuration and Profiles

Intermediate (6 topics)

07 Spring Data JPA

08 Entity Relationships

09 Validation and DTOs

10 Exception Handling

11 Database Migrations

12 Testing

Advanced (6 topics)

13 Spring Security

14 JWT and OAuth2

15 AOP and Caching

16 Actuator and Observability

17 Messaging: Kafka and RabbitMQ

18 Scheduling and Async

Master (4 topics)

19	Microservices with Spring Cloud
20	Docker, Kubernetes and CI/CD
21	Performance and Production Readiness
22	Real Project: E-commerce / Banking API

Beginner Level

01. Introduction to Spring and Spring Boot

Spring is a Java framework built around dependency injection. **Spring Boot** adds auto-configuration, starters and an embedded server so you can run an app with `java -jar`.

```
Generate a project at start.spring.io (Java 21, Maven, Spring Web)
```

02. Project Structure and First Application

`@SpringBootApplication` marks the entry point; `application.properties` configures the app.

```
@SpringBootApplication
public class App {
    public static void main(String[] args) { SpringApplication.run(App.class, args); }
}
```

03. Inversion of Control and Dependency Injection

The container creates beans and injects dependencies. Prefer constructor injection.

```
@Service
class OrderService {
    private final PaymentService payments;
    OrderService(PaymentService payments) { this.payments = payments; }
}
```

04. Beans and Stereotype Annotations

`@Component`, `@Service`, `@Repository`, `@Controller`, `@Configuration` + `@Bean` register beans.

```
@Configuration class Config { @Bean Clock clock() { return Clock.systemUTC(); } }
```

05. REST Controllers

`@RestController`, `@GetMapping`, `@PostMapping`, `@PathVariable`, `@RequestParam`, `@RequestBody`.

```
@RestController @RequestMapping("/api/users")
class UserController {
    @GetMapping("/{id}") User get(@PathVariable Long id) { return service.find(id); }
}
```

06. Configuration and Profiles

Use `application.yml`, `@Value`, `@ConfigurationProperties` and profiles (`dev`, `prod`).

```
server.port=8081
spring.profiles.active=dev
```

Intermediate Level

07. Spring Data JPA

Map entities with `@Entity` and use repositories for CRUD without SQL.

```
@Entity class User { @Id @GeneratedValue Long id; String name; }
interface UserRepo extends JpaRepository<User, Long> { List<User> findByName(String name); }
```

08. Entity Relationships

`@OneToMany`, `@ManyToOne`, `@ManyToMany`; lazy vs eager loading; avoid N+1 queries with fetch joins.

```
@OneToMany(mappedBy = "author") List<Post> posts;
```

09. Validation and DTOs

Validate input with Bean Validation (`@NotBlank`, `@Email`, `@Valid`); expose DTOs/records, not entities.

```
record CreateUser(@NotBlank String name, @Email String email) {}
```

10. Exception Handling

Centralise errors with `@RestControllerAdvice` and `@ExceptionHandler`; use ProblemDetail.

```
@ExceptionHandler(NotFound.class)
ResponseEntity<String> nf(NotFound e) { return
ResponseEntity.status(404).body(e.getMessage()); }
```

11. Database Migrations

Manage schema with Flyway or Liquibase instead of `ddl-auto=update`.

```
src/main/resources/db/migration/V1__create_users.sql
```

12. Testing

`@SpringBootTest`, `@WebMvcTest`, MockMvc, Mockito and Testcontainers.

```
mockMvc.perform(get("/api/users/1")).andExpect(status().isOk());
```

Advanced Level

13. Spring Security

Authentication and authorisation with filters, `SecurityFilterChain`, password encoders, method security.

```
@Bean SecurityFilterChain chain(HttpSecurity http) throws Exception {
    return http.authorizeHttpRequests(a ->
a.requestMatchers("/public/**").permitAll().anyRequest().authenticated()).build();
}
```

14. JWT and OAuth2

Stateless APIs with JWT, resource server and login with Google/GitHub via OAuth2.

```
spring.security.oauth2.resourceserver.jwt.issuer-uri=https://issuer
```

15. AOP and Caching

Cross-cutting concerns with `@Aspect`; caching with `@Cacheable` and Redis.

```
@Cacheable("users") public User find(Long id) { /* ... */ }
```

16. Actuator and Observability

Health, metrics and info endpoints, Micrometer, Prometheus, tracing.

```
management.endpoints.web.exposure.include=health,metrics,prometheus
```

17. Messaging: Kafka and RabbitMQ

Event-driven communication with Spring Kafka/AMQP; producers, consumers and retries.

```
@KafkaListener(topics = "orders") void on(String msg) { }
```

18. Scheduling and Async

`@Scheduled`, `@Async`, virtual threads support.

```
@Scheduled(cron = "0 0 * * * *") void hourly() { }
```

Master Level

19. Microservices with Spring Cloud

Service discovery, API gateway, config server, resilience (circuit breakers), distributed tracing.

```
Spring Cloud Gateway + Eureka + Resilience4j
```

20. Docker, Kubernetes and CI/CD

Build images with buildpacks (`spring-boot:build-image`), deploy to Kubernetes, automate with GitHub Actions/GitLab CI.

```
./mvnw spring-boot:build-image
```

21. Performance and Production Readiness

Connection pools, caching, pagination, indexes, GraalVM native images, graceful shutdown.

```
server.shutdown=graceful
```

22. Real Project: E-commerce / Banking API

Layered REST API: JPA + PostgreSQL, security with JWT, validation, tests, Docker, CI/CD.

Interview Question Bank

Q1. What is Spring Boot?
A framework on top of Spring providing auto-configuration, starters and an embedded server for production-ready apps quickly.

Q2. What does `@SpringBootApplication` do?
Combines `@Configuration`, `@EnableAutoConfiguration` and `@ComponentScan`.

Q3. What is dependency injection?
The container supplies a class's dependencies; constructor injection is recommended.

Q4. `@Component` vs `@Service` vs `@Repository`?
All register beans; the latter two add meaning, and `@Repository` also translates persistence exceptions.

Q5. What is auto-configuration?
Spring Boot configures beans based on classpath libraries and properties.

Q6. `@RestController` vs `@Controller`?
`@RestController` = `@Controller` + `@ResponseBody` (returns data, not views).

Q7. How do you avoid the N+1 problem?
Use fetch joins, entity graphs or batch fetching.

Q8. What is a bean scope?
Lifecycle of a bean: singleton (default), prototype, request, session.

Q9. How do you secure a REST API?
Spring Security with JWT/OAuth2, HTTPS, validation, role-based access.

Q10. What are Actuator endpoints?
Production endpoints exposing health, metrics and runtime information.

Capstone Project Ideas

1. Employee management REST API
2. Blog with JWT auth
3. Banking microservices
4. E-commerce order service with Kafka
5. URL shortener with Redis